

객체지향프로그래밍및실습
3차 과제

주차관리 시스템
-객체지향 버전-

소프트웨어학과
202322341
하세가와 레오(HASEGAWA REO)

<소개>

이번 과제는 2 차 과제와 동일한 객체지향 설계를 기반으로, A 대학교의 주차 관리 시스템을 개선하는 것이다. 입차 및 출차 기능을 관리하고, 차량의 종류에 따라 주차 요금을 자동으로 계산하는 시스템을 구현하였다. 이번 과제에서는 기존의 기능을 확장하여, 차량의 종류(가솔린, 하이브리드, 버스, 트럭)에 따른 속성 정보(배기량, 최대 승객 수, 중량)를 반영한 요금 계산 기능이 추가되었다. 이를 통해 보다 정교한 주차 요금 시스템을 구현하였다.

시스템의 핵심 명령어는 다음과 같다:

- e: 차량 입차
- x: 차량 출차
- v: 현재 주차 중인 차량 목록 보기
- i: 특정 일의 총 수입 계산
- r: 정기 차량 등록
- a: 등록된 정기 차량 목록 보기

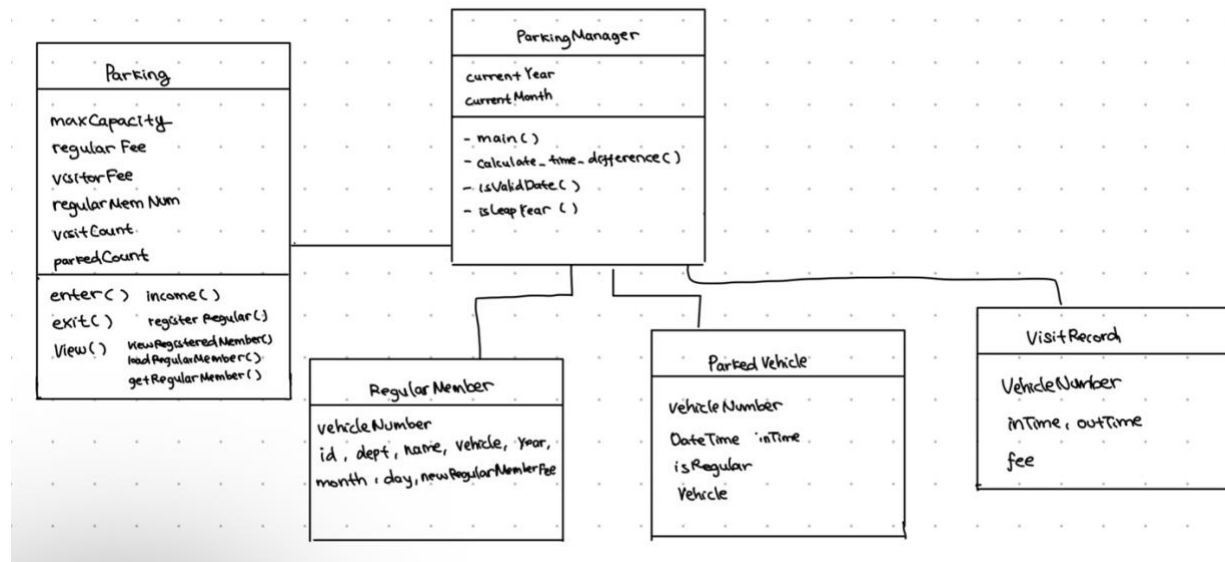
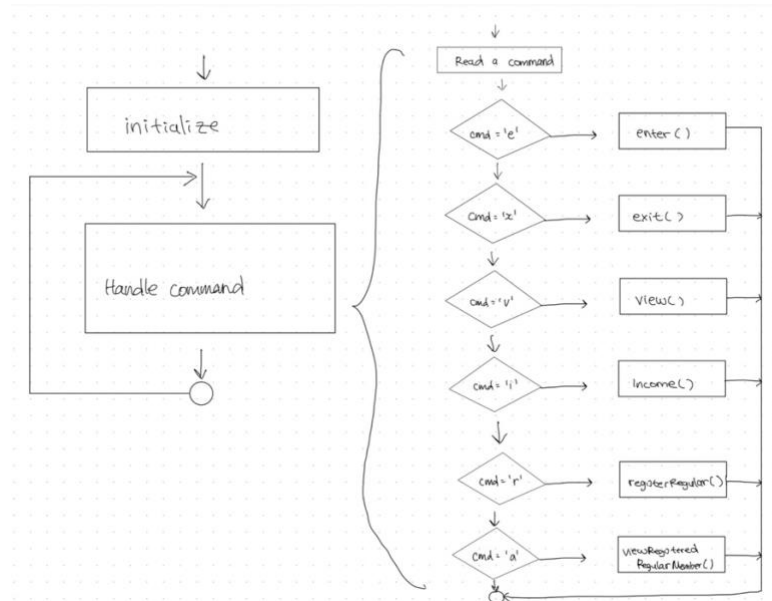
이외에도, 동일 차량의 중복 입차를 방지하는 기능과 주차 공간 초과 시 입차를 제한하는 기능도 구현하였다. 이번 과제의 가장 핵심적인 부분은 상속과 다형성을 활용하여 차량 클래스를 유연하게 설계하고, 차량 종류에 따라 상이한 요금 계산 로직을 오버라이딩을 통해 구현한 것이다. 상위 클래스인 Vehicle 에서 공통 속성과 메서드를 정의하고, 이를 GasCar, HybridCar, Bus, Truck 클래스들이 상속받아 각 차량의 특성에 맞게 동작하도록 구현함으로써, 객체지향적인 설계 원칙을 효과적으로 적용하였다.

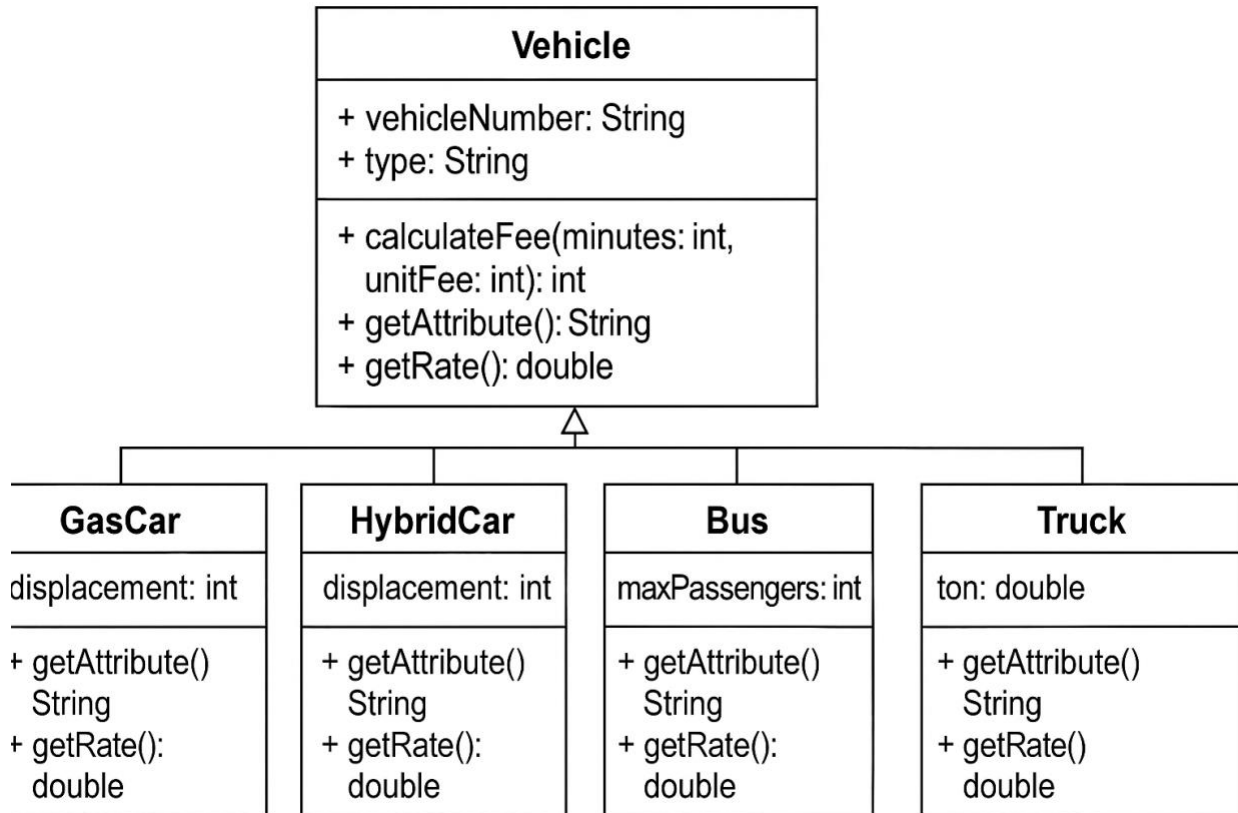
제대로 구현하지 못한 부분

항목	설명
없음	없음

<분석/설계>

-설계-





➤ 상속

1. Vehicle Class

모든 차량의 공통 속성 (vehicleNumber, type)과 공통 메서드 (calculateFee(), getAttribute(), getRate())를 정의한 추상 클래스.

2. Subclass 들의 상속 구조

Subclass	상속받은 클래스	고유 속성	설명
GasCar	Vehicle	displacement: int	배기량 기준 요금 계산
HybridCar	Vehicle	displacement: int	배기량 기준 요금 계산
Bus	Vehicle	maxPassenger: int	최대 승객 수 기반 요금 계산
Truck	Vehicle	ton: double	중량 기준 요금 계산

➤ 다형성

1. Method Overriding

Vehicle class 에 선언된 추상 메서드들은 모든 서브 클래스에서 구체적으로 오버라이딩 되었다.

- CalculateFee(int minutes, int unitFee): 차량별 요율을 적용하여 요금 계산 방식을 다르게 정의
- getAttribute(): 출력 시 각 차량의 속성 정보를 다르게 표현한다
- getRate(): 각 차량의 요금 계산 로직을 다르게 구현한다.

2. Overloaded

Enter() 메서드는 Overloading 을 활용하여 정기주차 차량용과 방문 차량용 두 가지 버전으로 나누어 구현하였다. 방문 차량의 경우에는 차량의 종류(type)와 속성(attribute)정보를 추가로 입력 받아, 해당 정보에 따라 GasCar, HybridCar, Bus, Truck 등의 객체를 생성한다.

<구현>

enter() 메서드 오버로딩 및 HybridCar 분기 처리할 때 정기 차량과 방문 차량의 입차 처리를 구분하기 위해 enter() 메서드를 오버로딩해야 한다. 방문 차량의 경우 차량 유형(g, h, b, t)과 속성값을 기반으로 GasCar, HybridCar, Bus, Truck 중 하나의 객체를 생성해야 하고 이 과정에서 특히 하이브리드 차량에 대해서는 요금이 절반만 부과되어야 하므로, 수입 계산 시 if (v instanceof HybridCar) 조건문을 통해 해당 차량이 하이브리드인지 확인하고, 요금을 절반으로 계산하는 분기 처리가 필요했다. 이러한 객체 생성 방식과 타입 분기에 대한 설계가 처음에는 익숙하지 않아 어려움을 겪었고, ChatGPT 의 도움을 받아 instanceof 연산자를 사용하는 적절한 시점과 메서드 구조를 파악할 수 있었다. 결과적으로 각 차량 타입에 맞는 요금 정책이 정확히 반영되도록 구현할 수 있었다.

<실행 결과>

1) 정기주차 차량 입차

```
● (base) hasegawareo@hasegawareisakuras-MacBook-Pro HW3_202322341 % javac ParkingManager.java
○ (base) hasegawareo@hasegawareisakuras-MacBook-Pro HW3_202322341 % java ParkingManager
> e 1111 2025 3 16 13 30
정기주차 차량 1111가 (이) 입차하였습니다!
> █
```

설명) 차량 번호 1111 이 파일 parking.txt 에 있는지 확인, 있으면 그 차량은 정기주차 차량이다.

2) 방문주차 차량 입차

```
> e 4444 2025 3 16 14 0 g 2000
방문주차 차량 4444가 (이) 입차하였습니다 !
> █
```

설명) 차량 번호 4444 는 파일 parking.txt 에 존재하지 않기 때문에 방문주차 차량으로 분류된다. 그리고 이번 3 차 과제에서는 방문주차 차량이 입차할 때 차량의 종류와 속성을 입력해야 하게 되었다.

3) 이미 입차한 차량이 다시 입차하는 경우

```
> e 4444 2025 3 16 15 0 g 3000
방문주차 차량 4444는 (은 ) 이미 입차한 차량입니다 !
> █
```

설명) 차량번호 4444 는 이미 입차했기 때문에 다시 입차를 못하게 만들었다. 오류메시지로 “방문주차/정기주차 차량 (번호) 이미 입차한 차량입니다!”

4) 다른 차량 입차

```
> e 3333 2025 3 16 18 10
정기주차 차량 3333가 (이) 입차하였습니다 !
> █
```

5) 주차 차량 목록 보기

```
> v
정기주차 차량목록
  1 1111 2025 03 16 13 30 e11111 총무처 홍길동 가솔린 1500
  2 3333 2025 03 16 18 10 s33333 사이버보안학과 을지문덕 하이브리드 2400
방문주차 차량목록
  1 4444 2025 03 16 14 00 가솔린 2000
> █
```

설명) v command 으로 통해 주차 차량 목록을 볼 수 있다(지금 주차하고 있는 차량 정보). 정기주차 차량과 방문주차 구분해 볼 수 있다. 지금까지 3 개의 차량의 입차를 시켰기 때문에 3 개의 정보가 출력되었다. 이번 3 차 과제에서는 추가로 차량의 종류와 속성 정보도 목록에 출력할 수 있게 만들었다.

6) 정기주차 차량 출차

```
> x 1111 2025 3 16 18 30
정기주차 차량 1111가 (이) 출차하였습니다 !
> █
```

설명) 정기주차 차량이 출차 할 때는 이미 학기 별로 주차요금을 냈기 때문에 따로 주차요금을 낼 필요가 없어서 주차요금 계산은 없다.

7) 방문주차 차량 출차

```
> x 4444 2025 3 16 18 45
방문주차 차량 4444가 (이) 출차 하였습니다 !
주차시간 : 285분
주차요금 : 34800원
> █
```

설명) 방문주차 차량은 정기주차 차량 처럼 학기별 주차요금을 내는 것이 아니기 때문에 10 분당의 주차요금 계산이 필요하다. 방문주차 차량의 경우

기본요금: $\text{ceiling}(\text{주차시간}/10 \text{ 분}) * 10 \text{ 분 주차 요금}$

- 가솔린 차 : 기본요금 x 배기량 요율
- 하이브리드 차 : 가솔린 차의 1/2
- 버스 : 기본요금 x 승객수 요율
- 트럭 : 기본요금 x 중량 요율

배기량 요율

배기량 < 1000 : 0.8
1000 <= 배기량 < 2000 : 1.0
2000 <= 배기량 < 3000 : 1.2
3000 <= 배기량 : 1.5

- 승객수 요율

승객수 < 12 : 1.0
12 <= 승객수 < 20 : 1.2
20 <= 승객수 < 30 : 1.5
30 <= 승객수 : 2.0

- 중량 요율

중량 < 1 : 1.0
1 <= 중량 < 2 : 1.5
2 <= 중량 < 4 : 2.0
4 <= 중량 : 3.0

차량번호 4444 는 285 분 동안 주차했기 때문에

기본요금: $29 * 1,000 = 29,000$

그리고 차량번호 4444 의 정류는 가솔린이고 배기량이 2000 이기 때문에 기본요금을 1.2 로 곱한다.

주차요금 = $29,000 * 1.2 = 34,800 \text{ 원}$

8) 총수입

```
> i 2025 3
총 수입 (2025년 3월) : 62,800원
- 정기주차 차량 : 28,000원
- 방문주차 차량 : 34,800원
> █
```

설명) 명령어를 통해 2025년 3월의 주차 수입을 확인할 수 있다. 정기 주차 차량의 요금은 한 학기(6개월) 기준으로 60,000 원이므로, 한 달 요금은 10,000 원이다. 이 기본 요금에 차량의 속성에 따라 가중치를 곱해 계산한다. 현재 등록된 정기 주차 차량은 총 3 대이다: Car (gas) 배기량 1500cc (1 대), Car (gas) 배기량 2000cc (1 대), Car (hybrid) 배기량 2400cc (1 대). 따라서, 요금 계산은 다음과 같다:

$$10,000 \times 1.0 + 10,000 \times 1.2 + 10,000 \times 1.2 / 2 = 28,000 \text{ 원}$$

방문 차량은 1 대이며, 총 285 분을 주차하였다. 이 시간은 10 분 단위로 올림하여 290 분이 되고, 이전 예시에서 계산한 것처럼 34,800 원이 나온다.

따라서 총 수입은 62,800 원이다.

9) 정기주차 차량등록 (새로운 기능)

```
> r 5555 2025 6 17 e55555 교무처 강감찬 b 20
정기주차 차량 (이) 등록되었습니다.
```

설명) r command 으로 정기주차 차량을 추가할 수 있다.

이 예시는 파일에 있는 차량이 아니라, 새로 정기주차 차량을 등록한 경우이다. 이번 3 차 과제에서는 차량의 종류가 있기 때문에, 등록할 때 차량의 종류와 그 속성 정보를 함께 입력해야 한다.

10) 정기주차 등록차량 보기

```
> a
1 1111 2025-03-01 e11111 총무처 홍길동 가솔린 1500
2 2222 2025-03-01 s22222 소프트웨어학과 이순신 가솔린 2000
3 3333 2025-03-01 s33333 사이버보안학과 을지문덕 하이브리드 2400
4 5555 2025-06-17 e55555 교무처 강감찬 버스 20
> █
```

설명) 현재 시점에 등록된 모든 차량의 정보를 차량 번호 순으로 보여준다. 3 차 과제에서는 차량의 종류와 그 속성 정보도 함께 확인할 수 있다.

11) 새롭게 등록한 정기주차 차량 입차

```
> e 5555 2025 6 17 18 20
정기주차 차량 5555가 (이) 입차 하였습니다 !
> █
```

설명) 차량번호 5555 를 정기주차 차량으로 했기 때문에 입차 가능하게 되었다.

12) 주차 차량 목록 보기

```
> v
정기주차 차량 목록
  1 3333 2025 03 16 18 10 s33333 사이버보안학과 을지문덕 하이브리드 2400
  2 5555 2025 06 17 18 20 e55555 교무처 강감찬 버스 20
방문주차 차량 목록
> █
```

설명) 차량번호 1111, 4444 는 입차했는데 이미 출차하기 때문에 차량 목록에 없다.
아직 주차하고 있는 차량 번호 3333 과 5555 만 정기주차 차량목록에 있다.

13) 총수입

```
> i 2025 6
총수입 (2025년 6월) : 43,000원
- 정기주차 차량 : 43,000원
- 방문주차 차량 : 0원
> █
```

설명) 실행 예시를 보면, 6 월에는 입차한 방문 주차 차량이 없기 때문에 방문 주차 수입은 0 원이다. 또한, 6 월에는 차량 번호 5555 가 새로 등록되어 정기 주차 차량이 총 4 대가 되었다. 따라서 3 월 기준(정기 주차 차량 3 대)에서 1 대가 추가된 셈이고, 해당 차량은 최대 승객 수 20 명의 버스이므로 한 달 요금은 $10,000 \times 1.5 = 15,000$ 원이 된다. 결과적으로 6 월의 총 수입은 $28,000 \text{ 원} + 15,000 \text{ 원} = 43,000 \text{ 원}$ 이다.

-오류-

14) 명령이 유효하지 않은 경우

```
> t 2025 3 18 18 00
알 수 없는 명령입니다 .
> █
```

t 라는 명령이 없기 때문에 “알 수 없는 명령입니다” 라는 메시지가 나온다.

15) 입력된 시간정보가 바르지 않은 경우

```
> e 5555 2025 4 28 25 20
잘못 된 시간 정보입니다 !
```

설명) 24 식간에로 표시되어야 하는데 25 시를 입력했기 때문에 “잘못 된 시간 정보입니다!”가 나왔다.

16) 입차하지 않은 차량이 출차하는 경우

```
> x 8888 2025 3 16 18 00
입차 하지 않은 차량입니다 !
```

설명) 입차한 주차차량에 없기 때문에 “입차하지 않은 차량입니다”가 나왔다.

17) 같은 차량번호인 정기차량 번호 등록하기

```
> r 3333 2025 3 17 e33333 교무처 강감
이미 등록된 차량입니다 !
```

설명) 3333 은 이미 정기주차 차량에 있기 때문에 다시 등록할 수 없다.

<명령 실행 예시>

```
○ (base) hasegawareo@hasegawareisakuras-MacBook-Pro: ~ % java ParkingManager
> e 1111 2025 3 16 13 30
정기주차 차량 1111가 (이) 입차 하였습니다 !
> e 4444 2025 3 16 14 00 g 2000
방문주차 차량 4444가 (이) 입차 하였습니다 !
> e 4444 2025 3 16 15 00 g 3000
방문주차 차량 4444는 (은) 이미 입차한 차량입니다 !
> e 3333 2025 3 16 18 10
정기주차 차량 3333가 (이) 입차 하였습니다 !
> v
정기주차 차량 목록
1 1111 2025 03 16 13 30 e11111 총무처 총길동 가솔린 1500
2 3333 2025 03 16 18 10 s33333 사이버보안학과 물자문덕 하이브리드 2400
방문주차 차량 목록
1 4444 2025 03 16 14 00 가솔린 2000
> x 1111 2025 3 16 18 30
정기주차 차량 1111가 (이) 출차 하였습니다 !
> x 4444 2025 3 16 18 45
방문주차 차량 4444가 (이) 출차 하였습니다 !
주차시간 : 285분
주차요금 : 34800원
> i 2025 3
총수입 (2025년 3월) : 62,800원
- 정기주차 차량 : 28,000원
- 방문주차 차량 : 34,800원
> r 5555 2025 6 17 e55555 교무처 강감찬 b 20
정기주차 차량 (이) 등록되었습니다 . 등록 요금 : 15,000원
> a
1 1111 2025-03-01 e11111 총무처 총길동 가솔린 1500
2 2222 2025-03-01 s22222 소프트웨어학과 이순신 가솔린 2000
3 3333 2025-03-01 s33333 사이버보안학과 물자문덕 하이브리드 2400
4 5555 2025-06-17 e55555 교무처 강감찬 버스 20
> e 5555 2025 6 17 18 20
정기주차 차량 5555가 (이) 입차 하였습니다 !
> v
정기주차 차량 목록
1 3333 2025 03 16 18 10 s33333 사이버보안학과 물자문덕 하이브리드 2400
2 5555 2025 06 17 18 20 e55555 교무처 강감찬 버스 20
방문주차 차량 목록
> i 2025 6
총수입 (2025년 6월) : 43,000원
- 정기주차 차량 : 43,000원
- 방문주차 차량 : 0원
>
```

<결론>

이번 3차 프로그래밍 과제를 통해 객체지향 설계와 구현에 대한 이해를 더욱 심화할 수 있었다. 과제의 핵심은 단순히 차량 입출차를 기록하는 것을 넘어서, 다양한 차량 종류(가솔린 차, 하이브리드 차, 버스, 트럭)에 따라 서로 다른 요금 계산 방식과 속성 정보를 처리하는 시스템을 객체지향적으로 모델링하는 것이었다. 특히 이번 과제를 수행하면서 상속과 다형성의 중요성을 알 수 있었다. 기본 클래스인 Vehicle에 공통 속성과 메서드를 정의하고, 이를 기반으로 각 차량 유형별로 기능을 오버라이딩함으로써 유연한 코드 구조를 만들 수 있었다. 예를 들어, calculateFee() 메서드는 Vehicle 클래스에서 추상 메서드로 선언하고, 각 하위 클래스에서 구체적인 요금 계산 방식에 맞게 오버라이딩하였다. 이렇게 함으로써 새로운 차량 종류가 추가되더라도 기존 코드를 거의 수정하지 않고 새로운 클래스를 확장하는 것만으로 기능을 확장할 수 있게 되어, 유지보수성과 확장성 모두에서 뛰어난 구조를 구현할 수 있었다.

과제를 수행하면서 어려웠던 점은, 다양한 차량 속성(배기량, 중량, 승객 수)에 따라 요율을 적용해 요금을 계산하는 로직을 어떻게 각 클래스에 분리할지 고민하는 과정이었다. 처음에는 모든 요금 계산을 하나의 메서드에서 처리하려 했으나, 이는 유지보수가 어렵고 가독성이 떨어지는 구조였다. 이를 상속과 오버라이딩을 통해 차량별로 책임을 나누자 코드가 훨씬 간결하고 이해하기 쉬워졌다.

마지막으로, 이번 과제를 통해 객체지향 설계에 기반한 실질적인 프로그램 개발 경험을 쌓을 수 있었고, 코드 재사용성과 유지보수성을 높이는 방법에 대해서도 배울 수 있었다.